

智慧旅宿平台 — 每個數字是怎麼來的？

完整數值來源說明書

版本:2026-07-22 · 專案: C:\AI\Smart_accommodation_4 讀者設定:完全沒學過資料分析的人。全文不預設任何統計背景,遇到專有名詞一律先用生活化例子解釋。目標:畫面上出現的**每一個數字**,你都知道它從哪來、誰算的、可不可信。

第 0 章 · 怎麼看這份文件

0.1 三種數字,可信度完全不同

這是全文最重要的一件事。平台上的數字分成三種,混在一起看會出大錯:

標記	種類	意思	可信度	例子
● 真實	真實觀測	直接從原始資料抄下來或數出來的,沒有經過任何推測	100%	「未來 30 天訂房率 64%」 — 日曆上真的 30 天裡有 19 天被訂走
● 統計	統計整理	把很多筆真實資料做加總、平均、排序後得到的	高,但要注意樣本數	「同商圈同房型中位 43%」 — 把 257 間房的訂房率排隊,取中間那一間
● 推估	模型推估	電腦「猜」出來的,不是真的發生過的事	僅供參考	「預測空屋率 65%」 — 模型根據房源特徵猜的,可能猜錯

本平台的立場:● 真實 和 ● 推估 打架時,一律以 ● 真實 為準。理由很簡單:真實日曆是「已經發生的事實」,模型只是「猜測」。平台把這個原則做成了「體質 × 檔期四象限」(第 3.2.3 節)。

0.2 一個高中生也能懂的比喻

想像你是班導師,要判斷哪個學生需要關心:

- 真實 真實觀測 = 他這學期實際缺席幾天(翻點名簿就知道)
 - 統計 統計整理 = 全班平均缺席天數、他排第幾名
 - 推估 模型推估 = 你根據「他住得遠、爸媽都上班、上學期成績下滑」猜他未來會不會常缺席
- 三個都有用,但如果點名簿寫他全勤,你卻因為「猜他會缺席」去找家長談話,那就本末倒置了。這個平台就是為了避免這種本末倒置而設計的。

0.3 每個數值的說明格式

本文每個數字都用同一套格式說明:

畫面文字 → 白話意思 → 計算公式 → 負責程式 → 資料分類 → 注意事項

第 1 章 • 資料從哪裡來

1.1 原始資料(人類蒐集的,沒有經過本平台加工)

存在 `data/` 資料夾,全部是「別人已經公開或我們自己爬下來」的檔案。

檔案	內容	筆數	來源	蒐集時間
<code>listings_cleaned.csv.gz</code>	台北市 Airbnb 房源基本資料(價格、房型、座標、設施、評分…)	6,241 間	Inside Airbnb(公開資料集)	2025-09-30
<code>calendar.csv.gz</code>	每一間房、每一天的 可否預訂狀態	234 萬列	Inside Airbnb	2026-06-30
<code>reviews_cleaned.csv.gz</code>	住客評論全文	約 24 萬則	Inside Airbnb	2025-09-30
<code>591_taipei_*_rooms.csv</code>	591 租屋網台北物件(月租)	約 8 千筆	自行爬取	2026-07-18
<code>taipei_rooms_only_v14_*.csv</code>	Booking.com 台北旅店(日租)	約 3 千筆	自行爬取	2026-07-18
<code>ddroom_taipei_by_rooms.csv</code>	DD 租租網台北物件(月租)	約 1 千筆	自行爬取	2026-07-18
<code>臺北捷運車站出入口座標.csv</code>	捷運出入口位置	—	台北市政府開放資料	—
<code>台北市公車站牌.csv</code>	公車站位置	—	台北市政府開放資料	—
<code>台北市超商資料集_含經緯度_門牌級.csv</code>	超商位置 與門牌地址	—	台北市政府開放資料	—
<code>taipei_restaurants.csv / taipei_school.csv / taipei_clinics.json / 臺北市公園基本資料.json</code>	餐廳/學校/診所/公園位置	—	開放資料	—
<code>hotels_taipei_osm.csv</code>	旅館位置	—	OpenStreetMap	—

注意 第一個必須誠實說明的限制: Airbnb 房源資料是 **2025-09-30** 爬的,日曆是 **2026-06-30** 爬的,中間差了 **9 個月**。這代表模型看到的房源狀態(價格、評分)已經是 9 個月前的了。這件事會在第 5 章「前瞻驗證」再談一次,因為它直接決定了模型準不準。

1.2 加工資料(本平台的程式算出來的)

這些是「預先算好存起來」的檔案。為什麼要預先算?因為 `calendar.csv.gz` 有 234 萬列,如果每次有人打開網頁都重新算一次,伺服器記憶體會爆掉(Streamlit 免費版只有約 1GB)。

檔案	內容	由誰產生	執行方式
<code>dataset_multimodal.csv</code>	模型的 訓練教材 :5,849 間房 × 59 個特徵	<code>scripts/build_dataset_final.py</code> (+ POI/NLP 合併)	離線跑一次
<code>_predictions.csv</code>	每間房的 風險分數 (空屋率預測、高風險機率、警報等級)	<code>scripts/train_backend_models.py</code>	離線跑一次
<code>_calendar_metrics.csv</code>	每間房的 365 天訂房狀態壓縮字串 + 各種檔期指標	<code>scripts/build_calendar_features.py</code>	離線跑一次
<code>_calendar_market.csv</code>	各行政區 × 房型 × 月份的市場訂房率	同上	離線跑一次
<code>_absa_listing.csv</code> / <code>_absa_market.csv</code> / <code>_absa_examples.csv</code>	評論拆成 14 個面向 的好評/負評統計	<code>scripts/build_absa.py</code>	離線跑一次
<code>_nlp_extra.csv</code>	每間房的 負評比例	<code>scripts/train_backend_models.py</code>	離線跑一次
<code>_model_metrics.json</code>	模型成績單	同上	離線跑一次
<code>models/backend_models_v2.joblib</code>	訓練好的 模型本體	同上	離線跑一次
<code>models/forward_validation.json</code>	用真實未來資料檢驗模型的成績	<code>scripts/build_calendar_features.py</code>	離線跑一次
<code>models/competitor_index.pkl</code>	四平台競品的 空間索引	<code>scripts/train_backend_models.py</code>	離線跑一次

1.3 三支預計算腳本在做什麼(白話版)

`scripts/build_calendar_features.py` — 把 234 萬列壓成 3.6 MB

原始日曆長這樣:

```
listing_id, date,      available
12345,      2026-07-01, f
12345,      2026-07-02, f
12345,      2026-07-03, t
...
```

`available` 只有兩個值: `t` = 可以訂, `f` = 不能訂。

程式做三件事:

1. **翻譯**: `f` → `1` (不能訂), `t` → `0` (可以訂)
2. **壓縮**: 把一間房 365 天的 0/1 接成一條字串 → `"1110001111..."`, 365 個字元
3. **統計**: 從這條字串算出各種指標(30 天訂房率、空檔天數、最長連續空檔...)

原本 234 萬列 → 變成 5,614 列(每間房一列), 檔案 3.6 MB。網頁載入只要不到一秒。

● 這裡有一個全平台最重要的資料陷阱, 請務必記住: Inside Airbnb 的 `available = 'f'` 有兩種完全不同的意思: ① 這天真的被客人訂走了 ② 房東自己把這天關掉(整修、自住、不想接客)

資料裡分不出來哪個是哪個。所以本平台講的「訂房率」嚴格說應該叫「不可訂率」。這個限制會在第 3.5 節、第 7 章再談, 並且影響了平台好幾處的設計。

`scripts/build_absa.py` — 把 24 萬則評論拆成 14 個面向

ABSA 全名 Aspect-Based Sentiment Analysis, 中文叫「面向情感分析」。白話說就是:

一則評論常常同時誇獎和抱怨: 「**早餐好吃但浴室很小**」。如果只算整篇是好評還壞評, 這則會被算成「中性」, 等於什麼都沒學到。面向分析要做的是把它拆成: 早餐 → 正面、空間大小 → 負面。

程式怎麼做(沒有用 AI 模型, 純粹是查字典):

1. 準備 14 本小字典, 例如「清潔」這個面向的關鍵詞有: 乾淨、整潔、髒、灰塵、打掃、`clean`、`dirty`、`dust`...
2. 掃過每一則評論, 看有沒有出現這些詞。有 → 這則評論談到「清潔」
3. **只看關鍵詞前後 30 個字**(叫「局部窗口」), 數這 30 字裡有幾個正向詞、幾個負向詞
4. 正向詞多 → 這則對「清潔」是好評; 負向詞多 → 是負評
5. 全部加總 → 每間房、每個面向有幾則好評、幾則負評

為什麼只看前後 30 字? 因為「早餐好吃但浴室很小」這句話, 「好吃」離「早餐」很近、離「浴室」很遠。用距離來判斷情感歸屬, 就能正確拆開。

為什麼不用 AI 語言模型? 因為 BERT 這類模型檔案要 400MB~1.1GB, Streamlit 免費版記憶體只有約 1GB, 一載入就當掉。查字典法雖然笨, 但可以離線重現、速度快、而且任何人都能驗證對錯。

`scripts/train_backend_models.py` — 訓練模型

第 5.1 節會詳細講。

第 2 章 • 首頁(`index.py`)

首頁是一張動畫, 底部有一列統計數字。

2.1 底部統計列

畫面文字	白話意思	怎麼算	資料分類
房源總數	資料庫裡有幾間房	數 <code>listings_cleaned</code> 有幾列(價格 200~80000 之間)	● 真實
評論總數	一共有幾則住客評論	數 <code>reviews_cleaned</code> 有幾列	● 真實
行政區數	涵蓋台北市幾個區	數 <code>neighbourhood_cleaned</code> 有幾種不同的值	● 真實
生活機能點位	捷運+公車+超商+餐廳+公園+學校+診所總數	把 7 個開放資料檔的列數加起來	● 真實

負責程式: `index.py` 的 `pick_listings()` **資料分類:** 全部是 ● 真實 真實觀測,就是單純「數數量」,沒有任何計算。

2.2 輪播的房源照片

怎麼挑的:每個行政區隨機抽 1 間,不足 36 張時再從剩下的房源隨機補。用固定的隨機種子(`seed=7`),所以每次打開看到的是同一批,不會亂跳。

點下去會顯示什麼:房源詳情 + AI 照片清晰度(見第 4.4 節)。

第 3 章 · 房東入口(本章最詳細)

程式: `pages/1_房東入口.py` (778 行)+ 8 個支援模組

這是整個平台的主頁面。畫面上方有 7 個分頁,順序是刻意安排的:**先看該處理哪一間 → 再看定價、口碑、檔期 → 模型診斷放後面**。因為模型準確度有限(第 5.2 節會誠實交代),所以不讓它當主角。

3.1 側邊欄(左側控制列)

3.1.1 「請選擇登入的房東」下拉選單

畫面顯示例如: `GNstay公館棧 (ID 38134494·81 間)`

欄位	怎麼來
房東名稱	<code>listings_cleaned</code> 的 <code>host_name</code> 欄位
ID	<code>host_id</code> 欄位
「81 間」	把 <code>_predictions.csv</code> 依 <code>host_id</code> 分組,數每組有幾列
「● 3」(若有)	該房東名下警報等級為紅色的房源數

排序方式:房源數由多到少。 **程式位置:** `pages/1_房東入口.py` 第 81~91 行 **資料分類:** ● 真實 真實觀測(單純分組計數)

3.1.2 「風險分類模型」單選(LightGBM / XGBoost)

這是切換要用哪個模型的分數。兩個模型是用**同一批資料、同一套特徵**訓練的,只是演算法不同,可以互相對照。切換後全站的風險數字會跟著換(`prob` ↔ `prob_xgb`)。

程式位置:第 94~103 行 **白話解釋:**就像同一份考卷給兩個老師改,看兩人給的分數差多少。差很多代表這題本身就模糊。

3.1.3 「附近比較半徑」滑桿(500~2000 公尺)

拖動它會改變「跨平台定價落點」「周邊地圖」「同商圈排名」的比較範圍。預設 1000 公尺。

3.2 分頁一:房源總表

 對應你截圖 1~4

3.2.1 上方五張 KPI 卡

以下用你截圖裡的房東(ID 38134494)做實際示範,每個數字我都用原始資料重算過,完全對得上。

卡片 1:名下房源 81 間

- **白話:**這位房東在平台上有 81 間房
- **公式:**數 `_predictions.csv` 裡 `host_id == 38134494` 的列數
- **程式:** `pages/1__房東入口.py` 第 158 行 → `len(MY)`
- **分類:** ● 真實 真實觀測

卡片 2:近 30 天訂房率 64%,副標「真實日曆」

- **白話:**這位房東名下所有房源,未來 30 天平均有 64% 的日子是「不可訂」的
- **公式:**

單一房源的 30 天訂房率 = 未來 30 天中「不可訂」的天數 ÷ 30
卡片顯示的 64% = 這位房東 81 間房的 30 天訂房率「平均」

- **實際驗算:**81 間房的 `booked_rate_d30` 平均 = 0.637 → 顯示 64%(四捨五入)
- **程式:**
 - 底層計算 → `scripts/build_calendar_features.py` 第 83~86 行(產出 `booked_rate_d30`)
 - 畫面顯示 → `pages/1__房東入口.py` 第 159~162 行
- **分類:** ● 真實 真實觀測 — 這是**直接從日曆數出來的**,不是模型猜的。副標特別寫「真實日曆」就是要跟旁邊那張 ● 推估 模型推估的卡片區隔。
- **注意** 注意:因為 `available='f'` 分不出「被訂走」和「房東關掉」,所以嚴格說這是「不可訂率」。若房東把某幾天關起來自用,也會被算進這 64%。

卡片 3:30 天空檔 758 天,副標「約 NT\$ 1,112,869」

- 白話:

- 758 天 = 這位房東 81 間房,未來 30 天內加起來有 758 個「空著沒人訂」的日子
- NT\$ 1,112,869 = 如果這些空檔全部填滿,以目前定價估算可以多賺這麼多錢

- 「空檔」的嚴格定義:未來 30 天內,連續 5 天以上都可訂且沒訂單的區段。零星空 1~2 天不算。

- 公式:

第一步(每間房):掃過 30 天的 0/1 字串,找出連續 5 個以上「0」的區段
gap_days_30d = 這些區段的總天數
第二步(加總):758 = 81 間房的 gap_days_30d 全部相加
第三步(金額):1,112,869 = $\sum$ (每間房的每晚價格 $\times$ 該房的 gap_days_30d)

- 實際驗算: $(\text{price} \times \text{gap_days_30d}).\text{sum}() = 1,112,869$ 可用 與截圖完全一致

- 程式:

- 找空檔 → scripts/build_calendar_features.py 的 find_gaps() 第 51~63 行
- 加總與金額 → pages/1__房東入口.py 第 154155~163164 行

- 分類:● 真實 真實觀測(天數)+ ● 統計 統計整理(金額)

- 注意 注意:

1. 這個金額是機會成本上限,不是「你損失了這麼多錢」。它假設空檔 100% 都能填滿,現實不可能。
2. 為什麼用 5 天當門檻?因為零星 1~2 天的空檔是短租的正常現象(客人退房到下一組入住之間),不值得警示。
3. 為什麼是 30 天而不是 90 天?因為遠期的日曆很多房東還沒開放,遠期的「空檔」多半不是賣不掉,而是還沒開賣。30 天內空檔才是真訊號。

卡片 4:需優先處理 5 間,副標「檔期空且體質偏弱」

- 白話:這 5 間房「模型覺得有問題」而且「日曆上真的空著」,兩邊都示警,最該先處理

- 公式:數「四象限分類 = 真警報 真警報」的房源有幾間(定義見下一節)

- 實際驗算:5 間 可用

- 程式: modules/quadrant.py 分類 + pages/1__房東入口.py 第 156、165 行

- 分類:● 推估 + ● 真實 混合(這正是它的價值:兩種來源都指向同一間房,才叫「真」警報)

卡片 5:平均預測空屋率 47%,副標「模型推估・僅供參考」

- 白話:模型猜這 81 間房未來一年平均會空 47% 的日子

- 公式:81 間房的 vac_pred 取平均

- 實際驗算:0.469 → 47% 可用

- 程式: pages/1__房東入口.py 第 166~167 行

- 分類:● 推估 模型推估 — 副標直接寫「僅供參考」,是因為模型的前瞻準確度只有 AUC 0.632(第 5.2 節詳述)。

- **注意** 為什麼卡片 2 是 64% 訂房率、卡片 5 卻說 47% 空屋率? 這兩個數字**不是同一件事**,不能相加減:
 - 卡片 2:● **真實** 未來 30 天的**真實**不可訂率
 - 卡片 5:● **推估** 未來一年的**模型推估**空屋率
 - 時間範圍不同、資料來源不同、可信度不同。平台刻意把它們並排,就是要讓房東看到「短期真實」與「長期推估」的差距。

3.2.2 「預測空屋率」到底是什麼?(模型的學習目標)

這個概念貫穿全站,先講清楚。

Y_vacancy(空屋率)的定義:

$$Y_vacancy = availability_365 \div 365$$

availability_365 是 Airbnb 資料裡的一個欄位,意思是「未來 365 天中還可以被預訂的天數」。

舉例:一間房未來一年有 219 天還開放可訂 → $Y_vacancy = 219 \div 365 = 0.60$ → 空屋率 60%。

模型要做的事:看一間房的 59 個特徵(價格、房型、評分、周邊捷運數量、評論情感…),猜出這個 **Y_vacancy** 是多少。

為什麼是 0.6 當高風險門檻? 團隊實測過 0.7 和 0.6 兩種標籤:

標籤	高風險房源占比	在機率門檻 0.6 時的表現
$Y \geq 0.7$	31.4%	精確率 0.71 / 召回率 0.23
$Y \geq 0.6$	37.4%	精確率 0.70 / 召回率 0.27

選 0.6,因為在精確率幾乎沒掉的情況下,能多抓到一些(召回率 0.23 → 0.27)。 **來源:** doc/01_資料分析報告
_threshold06.md

名詞小教室:精確率 vs 召回率 想像你是醫生,要篩檢 100 個人裡的 30 個病人:

- **精確率 0.70** = 你說「有病」的人裡面,70% 真的有病(30% 是誤報,虛驚一場)
- **召回率 0.27** = 全部 30 個真病人裡,你只抓到了 27%(其餘 73% 漏掉了)

兩者天生互相拉扯:抓越多人就越多容易誤報。平台用「雙層警報」來平衡 —— 紅色層追求精確(少誤報)、黃色層追求召回(少漏抓)。

3.2.3 「體質 × 檔期」四象限(本平台最核心的設計)

截圖 對應你截圖 1 下半與截圖 2

畫面顯示: **真警報** 真警報 5 間 / **隱形危機** 隱形危機 20 間 / **注意** 靠降價撐住 3 間 / **可用** 健康 41 間 / **不足** 檔期資料不足 12 間(合計 81 **可用**)

這個設計要解決什麼問題？

平台有兩種判斷房源好壞的方法,但它們常常**互相矛盾**:

- **推估 體質**(模型給的警報等級:紅/黃/綠)
- **真實 檔期**(日曆上未來 90 天的真實訂房率)

實測兩者的相關係數只有 **0.063**(滿分 1.0)。白話說就是:**這兩個判斷幾乎完全無關**。

如果只是把兩個數字並排,房東會看到「模型說高風險,但我明明訂滿了」而不知所措。所以平台把兩者**交叉**成四個象限,直接告訴房東該怎麼辦。

分類規則(`modules/quadrant.py` 第 49~60 行):

第一步:判斷「體質」

體質差 = 模型警報等級是 紅色 或 黃色

體質佳 = 綠色

第二步:判斷「檔期」

檔期空 = 未來 90 天訂房率 < 20%

檔期滿 = 未來 90 天訂房率 ≥ 20%

第三步:交叉

檔期空 (<20%) 檔期滿 (≥20%)

體質差 (紅/黃) **真警報** 真警報 **注意** 靠降價撐住

體質佳 (綠) **隱形危機** 隱形危機 **可用** 健康

沒有日曆資料 → **不足** 檔期資料不足

四個象限的意義與該做什麼:

象限	優先序	意思	建議行動	為什麼
真警報 真警報	1	模型和日曆都說有問題	立即檢視定價與 LIME 痛點,同步啟動空檔促銷	兩個獨立來源都示警,誤判機率最低
隱形危機 隱形危機	2	模型說沒事,但日曆空著	查近期變動:競品降價、季節性淡季、日曆設定或照片失效	模型看的是 9 個月前的靜態特徵,抓不到最近才發生的變化
注意 靠降價撐住	3	日曆訂滿,但模型說體質弱	檢查單價與 RevPAR(而非入住率),確認沒有賠本衝量	訂滿不等於賺錢,可能是價格砍太兇
可用 健康	4	兩邊都好	維持現狀,持續觀察同商圈行情	—
不足 資料不足	5	這間房不在日曆資料裡	僅依模型體質評估判讀	房源資料與日曆是不同批次爬的,約 1,600 間對不上

「檔期資料不足 12 間」是怎麼回事? 房源資料爬於 2025-09-30、日曆爬於 2026-06-30,兩批的房源清單不完全一樣。有些房源在這 9 個月間下架了,或是新上架但沒被舊資料收錄。這 12 間就是對不上的。**這是資料本身的限制,不是程式錯誤**。

程式: `modules/quadrant.py` (102 行)、畫面呈现在 `pages/1_房東入口.py` 第 168~176 行

這個設計是整個專案我最想介紹的地方:它承認了「模型不夠準」這件事,並且用真實資料把模型的不足補起來,而不是假裝模型很準。

3.2.4 象限統計表(截圖 2)

欄位	怎麼來
象限	<code>quadrant.py</code> 的 <code>QUADRANTS</code> 字典寫死的中文標籤
房源數	依 <code>quadrant</code> 欄位分組計數
說明 / 建議行動	字典裡預先寫好的固定文字(不是 AI 生成的)

程式: `modules/quadrant.py` 的 `summary()` 第 95~102 行 分類:● 真實 計數 + 固定文案

3.2.5 房源卡片(截圖 3、4)

每張卡片有 6 個資訊。以你截圖裡的第一張卡片為例:

```
#1356177442322068221
中山區·整棟出租·$1,650/晚
[40% 圓環] ● 推估 觀察 預測空屋率 65% 真警報 真警報 90天實訂 1%
▲ 高於商圈 6pp
```

我用原始資料查證這一間:

```
id = 1356177442322068221
行政區 = 中山區 房型 = Entire home/apt(整棟出租)
price = 1650 prob = 0.40 vac_pred = 0.6501
tier = yellow booked_rate_d90 = 0.0111 dist_med = 0.5864
```

全部對得上 可用。逐項說明:

① 圓環中央的 40%

- 白話:模型認為這間房「屬於高風險族群」的機率是 40%
- 它不是空屋率! 這是最容易誤會的地方。40% 是機率,65% 才是空屋率預測。
- 公式:模型 B(分類器)輸出的校準後機率 $P(\text{空屋率} \geq 60\%)$
- 程式: `risk_ring()` 第 126~137 行(用 SVG 畫圓弧,弧長 = 機率)
- 分類:● 推估 模型推估

名詞小教室:什麼是「校準」? 沒校準的模型很像考試愛亂猜的學生:它說「我有 90% 把握」,但實際只對了 60%。校準(本平台用 Isotonic 等張迴歸)就是把模型的自信心「調正」,讓它說 70% 的時候,真的大約 70 次裡對 70 次。這很重要,因為平台的通知門檻是「機率 $\geq 60\%$ 就發警報」——如果機率不準,門檻就沒意義。

② 圓環顏色與 ● 推估 觀察 標籤

• 公式:

機率 $\geq 0.60 \rightarrow$ ● 高風險 (紅色)
機率 $\geq 0.35 \rightarrow$ ● 推估 觀察 (黃色)
機率 $< 0.35 \rightarrow$ ● 真實 安全 (綠色)

- 這間房機率 0.40, 介於 0.35 與 0.60 之間 $\rightarrow$ ● 推估 觀察 可用

• 為什麼設兩層?

- 紅色 0.60: 此層精確率約 70% — 報了大概率是真的, 適合直接發通知
- 黃色 0.35: 此層召回率約 70% — 涵蓋大部分真高風險房源, 適合當觀察名單

- 程式: `scripts/train_backend_models.py` 第 48 行定義門檻; `modules/feature_engineering.py` 第 158 行判斷等級

- 分類: ● 推估 模型推估

③ 預測空屋率 65%

- 白話: 模型猜這間房未來一年會空 65% 的日子
- 公式: 模型 A(LightGBM 迴歸) 預測的 `Y_vacancy`, 乘 100
- 程式: `_predictions.csv` 的 `vac_pred` 欄位
- 分類: ● 推估 模型推估

④ 真警報 真警報 標籤

- 公式: 體質 = yellow(差) 且 90 天訂房率 $1\% < 20\%$ (空) $\rightarrow$ 真警報 可用
- 分類: ● 推估 + ● 真實 混合

⑤ 90天實訂 1%

- 白話: 未來 90 天裡, 只有 1% 的日子是不可訂的 — 幾乎全空
- 公式: `booked_rate_d90` = 未來 90 天中不可訂天數 $\div 90 = 0.0111 \rightarrow 1\%$
- 程式: `scripts/build_calendar_features.py` 第 83~86 行
- 分類: ● 真實 真實觀測

這張卡片的故事: 模型只給 40% 機率 (黃色觀察, 不算嚴重), 但日曆顯示未來 90 天幾乎完全沒訂單。這就是為什麼平台要並列兩種資料 —— 只看模型會低估這間房的問題。

⑥ ▲ 高於商圈 6pp

- 白話: 這間房的預測空屋率, 比中山區所有房源的中位數高了 6 個百分點

• 公式:

差距 = 本房源 vac_pred - 該行政區所有房源 vac_pred 的中位數
= 0.6501 - 0.5864 = 0.0637 → 6.4 個百分點 → 顯示「6pp」

• 顯示規則:

- 差距 > +0.02 → ▲ 紅色「高於商圈 N pp」
- 差距 < -0.02 → ▼ 綠色「低於商圈 N pp」
- 中間 → ◆ 灰色「與商圈持平」

• 程式: trend_arrow() 第 140~147 行;中位數在第 5051 行計算

• 分類: ● 統計 統計整理(拿 ● 推估 模型預測去比較)

名詞小教室:什麼是「中位數」?為什麼不用平均? 把 10 個人的薪水排成一列,中位數就是站在正中間那個人的薪水。為什麼不用平均?因為只要有一個郭台銘走進來,「平均薪水」立刻暴衝,但那不代表大家變有錢了。中位數不會被極端值拉走,更能代表「一般情況」。房價、訂房率都有少數極端值,所以本平台**幾乎所有「市場基準」都用中位數**。

名詞小教室:什麼是「pp」? pp = percentage point(百分點)。從 60% 變成 66%,是「上升 6 個百分點」,不是「上升 6%」(那會是 63.6%)。講百分比的變化時混用這兩個會差很多,所以本平台一律標 pp。

⑦ 卡片排序

- **規則:**先依象限優先序(真警報 → 隱形危機 → 靠降價撐住 → 健康 → 資料不足),同象限內再依機率由高到低
- **程式:**第 184~185 行
- **設計理由:**讓房東從上到下處理就對了,不用自己判斷先做哪一間

⑧ 「篩選象限」單選鈕(截圖 3 上方)

點選後只顯示該象限的卡片。「全部」= 81 張。**程式:**第 178~182 行。

3.3 分頁二:定價情報(本專案主打)

截圖 對應你截圖 5~8

這是 Airbnb 後台**看不到**的資訊:隔壁的 591 月租、Booking 日租,換算成同一個單位後,你的定價站在哪。

3.3.1 為什麼要換算成「每人每晚」?

四個平台的計價方式完全不同,直接比是錯的:

平台	原始標價	問題
Airbnb	每晚 \$1,650	—
Booking.com	每晚 \$4,200	可以直接比
591 租屋網	每月 \$22,500	單位不同 ,月 vs 晚
DD 租租網	每月 \$18,000	同上

而且就算都換成每晚,「一間可住 4 人的房 \$2,000」和「一間可住 1 人的房 \$2,000」也不能比。

所以要換算兩次:

第一步(統一時間單位):

月租平台:每晚等效價 = 月租 ÷ 30

日租平台:每晚等效價 = 原價

第二步(統一人數單位):

每人每晚 = 每晚等效價 ÷ 可住人數

實例(你截圖 5 的房源):

我的房源:每晚 \$1,229,可住 2 人

→ 每人每晚 = $1229 \div 2 = \$614$ **可用** 與截圖一致

程式: `modules/market_data.py` 第 58~66 行(`_finalize()`) **分類:** ● 統計 統計整理

注意 這個換算的三個誠實限制(畫面上也有標註):

1. **沒有計入押金:**591 租屋通常要押兩個月,Airbnb 不用。實際成本被低估了。
2. **沒有計入管理費:**很多月租物件另收管理費。
3. **沒有計入最短租期:**591 物件常要求至少租一年,和「住三晚」根本是不同商品。

所以這個比較只能看**相對定位**(我比別人貴還便宜),不能當成「換平台就能賺這麼多」。

3.3.2 五張平台卡片(截圖 5)

我的每人每晚 \$614 (每晚 \$1,229)

AIRBNB 475 筆 (中位 \$658)

BOOKING.COM 50 筆 (中位 \$4,xxx)

591租屋網 447 筆 (中位 \$750)

DD租租網 33 筆 (中位 \$522)

「475 筆」怎麼數的?

用一個叫 **BallTree** 的空間索引,快速找出「以我的房源為圓心、半徑 1000 公尺內」的所有物件。

名詞小教室:BallTree 是什麼? 想像你要在 1 萬個點裡找出離你 1 公里內的點。笨方法是把 1 萬個點的距離全算一遍。BallTree 是先把地圖切成一層層的「球」,查詢時先問「這顆大球整個都離我超過 1 公里嗎?」是的話整顆跳過,不用逐點算。效果:1 萬個點的查詢從幾秒縮短到千分之幾秒。本平台用它做兩件事:找 1 公里競品、找最近的捷運/超商。

距離怎麼算? 用 Haversine 公式(球面距離),因為地球是圓的,不能用平面的畢氏定理:

距離 = $2 \times \text{地球半徑} \times \arcsin(\sqrt{(\sin^2(\Delta\text{緯度}/2) + \cos(\text{緯度}1) \cdot \cos(\text{緯度}2) \cdot \sin^2(\Delta\text{經度}/2))})$
地球半徑 = 6,371,000 公尺

「中位 \$658」:半徑內該平台所有物件的「每人每晚」排序後取中間值。

程式:

- 空間索引 → `modules/competitor.py` 的 `CompetitorIndex.query()` 第 24~33 行
- 各平台統計 → 同檔 `stats()` 第 42~48 行
- 畫面 → `pages/1__房東入口.py` 第 555~572 行

分類:●真實 計數 + ●統計 中位數

注意 為什麼 Booking 中位數(\$4,xxx)比別人高這麼多? 因為 Booking.com 上的多半是旅館,不是民宿;而且旅館房型常常標示「可住 2 人」但價格是商務旅館等級。這不是計算錯誤,是**商品本質不同**。看跨平台比較時要記得這件事。

3.3.3 「同容量層貴於 39%」(截圖 6 第一句)

畫面:「同容量層(1-2人)共 553 筆競品,你的每人每晚 **貴於 39%**,同層中位為 \$762。」

為什麼要分「容量層」? 因為大房子本來就比較貴,拿 8 人房跟 2 人房比毫無意義。所以先分三層:

可住 1~2 人 → 「1-2人」層
可住 3~4 人 → 「3-4人」層
可住 5 人以上 → 「5人以上」層

只跟**同一層**的競品比。

「貴於 39%」怎麼算?

把同層 553 筆競品的「每人每晚」全部排成一列,
數出有幾筆比我便宜,除以總數:

貴於比例 = (比我便宜的競品數) ÷ 553 = 39%

白話:**我比 39% 的同級競品貴,比 61% 的便宜** → 我算是中間偏便宜。

顏色規則: ≥60% 紅色(偏貴)、40~60% 黃色、<40% 綠色(偏便宜)。截圖顯示綠色 **可用**

保護機制:同層競品少於 5 筆就不計算(顯示「樣本不足」),避免用 2、3 筆資料下結論。

程式: `modules/competitor.py` 第 50~59 行 分類:●統計 統計整理

名詞小教室:什麼是「百分位」? 就是「排名的百分比版」。全班 50 人,你考第 20 名 → 你贏過 30 人 → 百分位 60%。好處是不管班上幾個人都能比。這裡的「貴於 39%」就是價格的百分位。

3.3.4 「設施缺口」(截圖 6 第二句)

畫面:「設施缺口:**廚房/可開伙**(周邊 64% 具備)、**工作空間**(周邊 59% 具備) —— 這些是同商圈的標配。」

怎麼算的:

第一步:把四個平台的設施描述,用關鍵詞比對成 14 種標準設施
(例:「冷氣」「air conditioning」「ac unit」→ 都歸成「冷氣」)

第二步:計算覆蓋率

某設施覆蓋率 = 1 公里內有這項設施的競品數 ÷ 競品總數

第三步:找缺口

覆蓋率 $\geq$ 50%(過半競品都有)且我沒有 → 列為「缺口」

取前 4 項顯示

14 種標準設施:冷氣、洗衣機、冰箱、電視、WiFi、電梯、車位、陽台、浴缸、熱水器、廚房/可開伙、自助入住、飲水機、工作空間

程式:

- 設施標準化 → `modules/market_data.py` 的 `_canon_amenities()` 第 39~44 行
- 覆蓋率 → `modules/competitor.py` 第 61~67 行
- 畫面 → `pages/1_房東入口.py` 第 584~591 行

分類:●統計 統計整理

注意 限制:設施是從**文字描述**比對出來的。如果房東沒在文案裡寫「有冷氣」,即使實際有,也會被判定成「沒有」。所以「缺口」有可能是「有但沒寫」。

3.3.5 同商圈同房型 真實已訂率基準(截圖 6 下半)

我的 90 天訂房率 100%
同商圈同房型中位 43% (257 間)
我的排名 贏過 94%
我的 30 天訂房率 100%

數值	公式	程式	分類
我的 90 天訂房率	該房源 <code>booked_rate_d90</code>	<code>build_calendar_features.py</code>	● 真實
同商圈同房型中位	同行政區 + 同房型 + 有日曆資料的房源, <code>booked_rate_d90</code> 取中位數	<code>pages/1</code> 第 598~609 行	● 統計
「257 間」	符合條件的房源數	同上	● 真實
我的排名「贏過 94%」	(訂房率比我低的房源數) ÷ 257	第 611~613 行	● 統計
我的 30 天訂房率	該房源 <code>booked_rate_d30</code>	同上	● 真實

保護機制:同商圈同房型少於 10 間就不建立基準。

注意 一個必須誠實說的事:這間房 90 天訂房率 100%,看起來超級好。但因為 `available='f'` 分不出「訂滿」和「房東封鎖」, **100% 也可能代表房東把未來三個月全部關掉了**(例如整修或轉長租)。資料無法區分。這正是第 3.5 節「未來檔期」要處理的問題。

3.3.6 訂房率分佈直方圖(截圖 7 上半)

圖在說什麼:橫軸是「未來 90 天訂房率」(0 到 1),縱軸是「有幾間房落在這個區間」。紅色虛線是「我的位置」。

怎麼畫的:把 257 間同商圈同房型的房源,依訂房率分成 20 個桶子,數每個桶子有幾間。

程式: `pages/1__房東入口.py` 第 616~626 行(用 Plotly 的 `histogram`) **分類:**● 真實 計數

名詞小教室:直方圖怎麼看?想像把全班身高從矮到高排好,每 5 公分放一個籃子,數每個籃子有幾個人。柱子越高代表那個身高範圍的人越多。這張圖告訴你:大部分房源集中在哪個訂房率、你在不在主流。

這張圖的判讀:圖上最左邊(訂房率 0~0.1)的柱子最高(約 46 間),代表**近五分之一的同類房源未來 90 天幾乎沒訂單**。市場整體不好,不是只有你。

3.3.7 周邊四張指標卡(截圖 7 下半)

1000M 內 AIRBNB	491 間
周邊高風險機率中位	37%
我的高風險機率 VS 周邊	42% (高於 58% 的鄰居)
同商圈排名(低風險優先)	第 288 / 492 名

數值	公式	分類
491 間	用 Haversine 算出 1 公里內的 Airbnb 房源數(不含自己)	● 真實
周邊高風險機率中位 37%	這 491 間的 <code>prob</code> 取中位數	● 統計 (基於 ● 推估)
我的 42%	本房源的 <code>prob</code>	● 推估
高於 58% 的鄰居	(機率比我低的鄰居數) ÷ 491	● 統計
第 288 / 492 名	把 492 間(含自己)依機率由低到高排,我排第 288	● 統計

程式:第 633~653 行 **注意** 注意:這四個數字全部建立在 ● 推估 模型機率上,所以它們繼承了模型的不確定性。排名第 288 不代表「你真的是第 288 差」,只代表「模型認為」。

3.3.8 周邊房源分佈圖(截圖 8)

這是自製的 Leaflet 互動地圖(不是 Plotly)。

為什麼要自己寫? Streamlit 的 Plotly 元件**不支援 hover(滑鼠移過)事件**,只支援 click。而需求是「滑鼠移到地圖點位,右側清單要同步標示」。所以把地圖和清單包成同一段 HTML 塞進 iframe,所有互動在瀏覽器裡完成,不用回傳給伺服器。

畫面上每個元素的來源:

元素	來源	分類
圓點位置	房源的經緯度	● 真實
圓點顏色	綠 = 預測空屋率 <40%、黃 = 40~69%、紅 = ≥70%	● 推估
黑色閃爍點	本房源(CSS 動畫 <code>@keyframes pulse</code>)	● 真實
藍色虛線圓	比較半徑(隨側邊欄滑桿改變)	—
圖層勾選框	591(357)、Booking(50)、DD 租租網(26)	● 真實 計數
右側清單「距離 4 公尺」	Haversine 距離,四捨五入到整數公尺	● 真實
右側清單地址	推估地址 (見下方說明)	● 統計 推估

注意 兩個看起來像錯誤、其實正常的現象(我特別查證過):

1. **「距離 4 公尺」是真的**。這位房東(GNstay 公館棧)81 間房大多在**同一棟大樓的不同房間**,所以隔壁房間距離就是幾公尺。我實際計算前 180 筆的距離分佈:最近 4 公尺、中位數 265 公尺、最遠 457 公尺 —— 合理。
2. **所有地址都一樣**。因為地址是「推估」的,不是真的。Airbnb 為了保護隱私**不公開門牌**,只給經緯度。平台的做法是:找出離該座標最近的、有門牌的超商或餐廳,取它的地址並去掉門牌號,只留路段。同一個街廓的房源當然會得到同一個路段名。

程式: `modules/geo_utils.py` 的 `nearest_address()` 第 284~313 行 這是「離線、免 API、無流量限制」的取捨 —— 精確度換速度與成本。

程式: `modules/map_view.py` (265 行) **技能包:**這個元件已經抽成可重複使用的 skill(`streamlit-interactive-map`)

3.3.9 同商圈風險分佈散點圖 + 排名表

散點圖:橫軸 = 每晚價格、縱軸 = 預測空屋率、點大小 = 可住人數、顏色 = 空屋率高低。黑色虛線 = 本房源定價。**排名表:**半徑內前 60 間,依高風險機率由低到高排,本房源前面加 → 標記。

程式:第 673~705 行 **分類:**● 推估 模型推估為主

3.4 分頁三:口碑情報(評論面向分析)

程式: modules/absa_sections.py

3.4.1 評論自動摘要

畫面上方藍色方框,一段 3~5 句的話,標註「規則版」或「LLM 生成」。

兩種版本:

版本	誰寫的	何時出現
規則版	Python 程式套模板組句	預設
LLM 版	Gemini 潤飾	按下「用 LLM 重寫摘要」才生成

關鍵設計:LLM 只負責把數字寫成通順句子,不負責產生數字。所有百分比都由前面的 ABSA 統計算好,塞進提示詞裡,LLM 只能引用不能瞎編。詳見 doc/07_LLM生成原理說明.md。

程式: modules/review_summary.py **分類:**● 統計 統計整理(數字)+ 文字生成(句子)

3.4.2 面向負評率長條圖

畫面:每個面向一條橫棒(本房源負評率)+ 一個菱形(市場基準)。

數值	公式	分類
本房源負評率	該面向負評則數 ÷ 該面向提及則數	● 統計
市場基準	同行政區(樣本不足則全市)該面向的負評率	● 統計
差距	本房源 - 市場	● 統計

保護機制:某面向提及少於 3 次就不列入,避免「1 則負評 = 100% 負評率」這種假訊號。

全市三大痛點(實際計算結果):空調冷氣 15.7%、隔音噪音 13.6%、空間大小 13.4%

程式: modules/absa_sections.py 的 render_listing_absa()

注意 限制:字典法會誤判。例如「不會吵」含有「吵」,可能被算成負面。局部窗口減輕了這個問題但沒有完全解決。
優點是可離線重現、任何人都能檢查,缺點是不如深度學習模型精準。

3.5 分頁四:未來檔期

截圖 對應你截圖 9~11

程式: `modules/calendar_sections.py` + `modules/calendar_analytics.py`

3.5.1 上方五張 KPI(截圖 9)

以截圖那間房為例(我查證過):

未來 365 天已訂率 49% (180 天)
未來 30 天已訂率 100%
未來 60 天已訂率 100%
未來 90 天已訂率 100%
90 天內空檔 0 天 (0 段·最長 0 天)

數值	公式	程式	分類
365 天已訂率 49%	365 天中不可訂天數 ÷ 365 = 180 ÷ 365	<code>build_calendar_features.py</code> 第 81 行	● 真實
「180 天」	不可訂的總天數	第 82 行	● 真實
30/60/90 天已訂率	各時窗內不可訂天數 ÷ 該時窗天數	第 83~86 行	● 真實
90 天內空檔	90 天內連續 5 天以上空檔的總天數	第 88~93 行	● 真實
「0 段·最長 0 天」	空檔區段數、最長區段天數	同上	● 真實

注意 這組數字本身就是最好的教材: 30/60/90 天都 100%,但 365 天只有 49%。

差異來自哪?我實際查了這間房的逐月訂房率:

2026-07 100%
2026-08 100%
2026-09 100%
2026-10 100%
2026-11 100%
2026-12 87%
2027-01 0% ← 從這裡開始全是 0
2027-02 0%
2027-03 0%
2027-04 0%
2027-05 0%

這不代表 2027 年沒人訂,而是房東還沒開放那幾個月的日曆。Airbnb 預設只開放未來 12 個月,而且很多房東只維護近期。所以:

「未來 365 天已訂率」這個數字對遠期是不可信的。真正該看的是 30 天與 90 天。

3.5.2 逐日訂房熱度圖(截圖 10 左)

畫面:格子狀日曆,橫軸是星期一到日,縱軸是第 1 週到第 13 週,紅色 = 已訂/不可訂,綠色 = 空房可訂。

怎麼來的:把 365 個字元的 0/1 字串攤開成 365 列,每列標上日期、星期幾、第幾週,再排成表格。

程式: modules/calendar_analytics.py 的 `daily_frame()` + calendar_sections.py 的 `_heatmap()` **分類:** ● 真實
真實觀測 — 這是全站最「原始」的一張圖,直接把原始資料畫出來,零加工。

3.5.3 未來 12 個月:本房源 vs 同商圈(截圖 10 右)

畫面:藍色柱子 = 本房源各月訂房率,虛線 = 同商圈同房型基準。

注意 這一區在 2026-07-22 做了重要修正,原因值得說明:

問題:遠期月份的市場基準完全不可信。我實際統計了全體房源的逐月訂房率分佈:

月份	完全 0% 的房源比例	完全 100% 的房源比例	中位數
第 2 個月	10%	—	55%
第 3 個月	46%	—	6%
第 4 個月	54%	—	0%
第 11 個月	36%	約 47%	100%
第 12 個月	36%	約 47%	100%

看出問題了嗎?遠期月份的房源**不是完全空著(0%)就是完全鎖死(100%)**,兩極各半,幾乎沒有中間值。

把這兩群平均起來會得到約 49% —— 但**沒有任何一間真實房源是 49%**。這個「基準」不對應現實中的任何東西。

拿它當基準,就會對每一間把遠期日曆開著的房源發出「你的訂房率 0%,低於基準 49 個百分點,建議促銷」的**假警示** —— 但那間房根本還沒開賣。

修正後的做法:

1. 市場基準只採計「該月既非全空、也非全滿」的房源 — 也就是日曆確實在營運的房源
2. 該月可判讀房源比例低於 35% → 整月標為不可判讀(灰色柱子,不下結論)
3. 超出可信視窗(預設 6 個月)→ 不下結論
4. 本房源該月是 0% 或 100% → 不下結論(無法區分是沒人訂還是沒開賣/封鎖)
5. 只在「可判讀」月份中挑選最差月份發出建議

修正後的實際效果(以截圖那間房為例):

月份	本房源	基準	可判讀	原因
2026-07	100%	38%	● 否	本房源此月完全不可訂 — 可能是房東主動封鎖
2026-09	100%	27%	● 否	市場僅 17% 的房源此月日曆在營運
2027-01	0%	33%	● 否	超出可信視窗(第 8 個月)

→ 0 個可判讀月份,系統不再發出任何假警示,改為顯示「本房源目前沒有可判讀的月份 —— 這代表資料不足以下結論,不代表經營有問題」。

程式: `modules/calendar_analytics.py` 的 `monthly_baseline()`、`monthly_vs_market()`;畫面在 `calendar_sections.py` 分類: ● 統計 統計整理(含可信度標記)

這一段是我最想讓人看到的部分:一個數字看起來很合理(「低於基準 49 個百分點」),但追到底層才發現那個基準根本不存在於現實。做資料分析,重點不是算出數字,而是知道數字什麼時候不能用。

3.5.4 空檔警示表

畫面:起日、迄日、連續天數、建議。

建議規則(寫死的,不是 AI):

連續 ≥ 21 天 $\rightarrow$ ● 長空檔:建議大幅折扣或開放長租
連續 ≥ 10 天 $\rightarrow$ ● 推估 中空檔:建議限時折扣 10~15%
其餘(5~9 天) $\rightarrow$ ● 真實 短空檔:可設最後一分鐘折扣

程式: `calendar_sections.py` 第 123~126 行 分類:● 真實 天數 + 固定規則

3.5.5 營收最適定價(截圖 11)

畫面:

建議價格帶	\$3,416	(調高 \$2,187)		
該價格帶年營收估算	\$427,809	(平均已訂 125 天 · 38 筆同儕)		
價格中位	已訂天數	已訂率	年營收估算	樣本數
\$371	133.0	37%	\$49,473	38
\$685	191.0	52%	\$130,468	38
\$860	119.0	33%	\$102,554	38
...				

這張表是怎麼做出來的(五步):

第一步:找同儕

同行政區 + 同房型 + 有日曆資料的房源
(若少於 30 間 → 放寬為「同房型全台北市」)

第二步:去極端值

剔除價格最高的 2%(避免少數天價房源扭曲結果)

第三步:分成 8 個價格帶

用 qcut(等頻分箱)—— 每個價格帶放進「一樣多」的房源
所以每帶樣本數都是 38 左右

第四步:算每帶的平均表現

價格中位 = 該帶房源價格的中位數
已訂天數 = 該帶房源「未來 365 天實際不可訂天數」的平均
已訂率 = 已訂天數 ÷ 365

第五步:估算年營收

年營收估算 = 價格中位 × 已訂天數

驗算第一列: $\$371 \times 133.35 \text{ 天} = \$49,473$ 可用 (表格顯示的 133.0 是四捨五入,實際 133.35)

名詞小教室:什麼是「等頻分箱」(qcut)? 有兩種分組方式:

- **等寬:**每 1000 元一組(\$0-1000, \$1000-2000...)→ 有些組可能一間房都沒有
- **等頻:**不管價格,每組都放一樣多間房 → 組的價格範圍會不一樣寬

這裡用等頻,是為了讓每個價格帶的樣本數相當(都是 38 筆),比較才公平。

「建議價格帶 \$3,416」怎麼選的:在 8 個價格帶中,挑「年營收估算」最高的那一帶,取它的價格中位數。

程式: modules/calendar_analytics.py 的 peer_revenue_curve() 第 136-167 行 → optimal_price() 第 170-177 行

分類: ● **真實** 已訂天數是真實的 + ● **統計** 分組統計

注意 三個必須說清楚的限制(畫面下方也有標註):

1. **這是橫斷面比較,不是因果關係。**意思是:「同商圈裡,定價 \$3,416 附近的房源,平均年營收最高」。**不等於**「你把價格從 \$1,229 調到 \$3,416,就會賺 \$427,809」。因為貴的房源本來就可能地段更好、裝潢更好、照片更漂亮。**真正的價格彈性要做 A/B 調價實驗才知道。**
2. **這個方法有系統性偏誤:它幾乎總是叫你漲價。**我實際跑了三個商圈驗證:

商圈・房型	營收最高的價格帶
中山區・整棟出租	最高價帶(\$7,215)
中山區・私人套房	最高價帶(\$3,217)
大安區・整棟出租	第 7 帶(\$6,994,共 8 帶)

原因: 年營收 = 價格 × 已訂天數,而在橫斷面上,貴的房源「已訂天數」並沒有明顯比較少。所以價格越高,乘出來的營收就越高。**所以這個建議應該當成「你的定價相對市場偏低」的訊號,而不是「照著調就對了」的指令。**

3. 「已訂天數」同樣受 `available='f'` 二義性影響 — 封鎖日曆的房源會被誤算成「訂很滿」。

3.6 分頁五:風險診斷

 對應你截圖 12~13

3.6.1 左側:What-if 模擬 + 風險環(截圖 12 左)

[每晚房價滑桿] 1229

[最低入住天數] 30

GroupKFold 00F 誠實預測

[42% 圓環] ● 推估 觀察

環 = 高風險機率 $P(\text{空屋率} \geq 60\%)$, 紅 $\geq 60\%$ ·黃 $\geq 35\%$ ·綠 $< 35\%$

預測空屋率(模型A): 67%

模型: LightGBM·完整變體

「GroupKFold 00F 誠實預測」是什麼意思?(這是本平台最重要的技術誠實聲明)

先講一個容易犯的錯:

老師出 100 題給學生練習,考試又出**同樣的 100 題**。學生考 95 分,但這不代表他學會了——他可能只是**背下答案**。

機器學習也一樣。如果模型在「訓練用過的資料」上測分數,分數會虛高。

更隱蔽的陷阱:同一個房東的房源會互相洩漏答案。一個房東有 81 間房,經營手法、定價策略、照片風格都很像。如果訓練時看過其中 50 間,測試時考另外 31 間,模型可以靠「認出這是同一個房東」來作弊。

GroupKFold 就是防這個的:

把 5,849 間房依「房東」分成 5 組,確保同一個房東的房源一定在同一組裡,絕不會被拆開。

然後輪流:用 4 組訓練 → 考剩下那 1 組 → 換一組 → 重複 5 次

這樣每一間房都被「沒看過它房東」的模型考過一次。

這個分數叫 00F (Out-Of-Fold, 袋外預測), 才是誠實的分數。

兩種評估法的實際差距(`models/eval_results.json`):

評估方式	迴歸 R^2	說明
單次隨機切分(不分房東)	較高	虛高 , 含房東洩漏
GroupKFold(依房東)	0.243	誠實

全站顯示的每一個風險數字,都是 **00F 誠實分數**,不是模型在自己看過的資料上的分數。

數值	公式	程式	分類
圓環 42%	OOF 校準機率 <code>P(空屋率≥60%)</code>	<code>_predictions.csv</code> 的 <code>prob</code>	● 推估
● 推估 觀察	$0.35 \leq 0.42 < 0.60$	<code>feature_engineering.py</code> 第 158 行	● 推估
預測空屋率 67%	OOF 迴歸預測 <code>vac_pred</code>	同上	● 推估
「完整變體」	這位房東有多間房 → 用含房東特徵的完整模型	第 149 行	—

「完整變體 vs 冷啟動變體」：

完整模型 (59 特徵)：給有經營歷史的房東，包含「房東接受率、回覆速度、經營天數」等 7 個房東身分特徵

冷啟動模型 (52 特徵)：給只有 1 間房的新房東，移除那 7 個特徵
因為新房東根本沒有這些歷史紀錄

判斷方式：`calculated_host_listings_count ≤ 1` → 走冷啟動。

What-if 模擬怎麼運作？

拖動「每晚房價」滑桿後，程式做的事：

第一步：複製這間房的資料
 第二步：把 `price` 改成新價格
 第三步：連動更新「跟價格有關」的其他特徵：
 `price_per_person` = 新價格 ÷ 可住人數
 `price_per_bedroom` = 新價格 ÷ 臥室數
 `price_pctl_nbhd` = 新價格在「同區同房型真實價格分佈」中的百分位
 第四步：重新丟給模型預測
 第五步：算出差異，套回 OOF 基準值

第三步的 `price_pctl_nbhd` 是關鍵。這是 v4 版修正的重點：

樹模型學到的價格影響，**主要不在絕對價格上，而在「你比同區多少人貴」這個相對排名上**。如果只改絕對價格而不重算排名，模型幾乎不會有反應，使用者就會抱怨「我調了價格風險卻不動」。

程式：`modules/feature_engineering.py` 的 `simulate_price_change()` 第 188~215 行

注意 為什麼有時候調了價格風險還是不動？畫面上會出現一段說明，原因有兩個：

1. **樹模型是階梯函數**。它的判斷是「價格 > 2000 嗎？」這種二分法，價格從 1200 調到 1300 可能還在同一階，輸出完全一樣。
2. **在這份資料裡，價格根本不是主要風險因子**。前向特徵選擇中，價格排第 19 名、最低入住天數排第 15 名，都在「噪音帶」。真正的強訊號是**房東接受率、回覆速度、周邊口碑排名**。

這是誠實的發現，不是程式壞掉。平台選擇把它寫在畫面上告訴使用者，而不是假裝有效果。

3.6.2 右側:LIME 原因 Top 3(截圖 12 右上)

1. 近期平均最短入住 > 20.80 +11.1 pp
[紅色長條]
可用 加分項:房東房源數 > 29.00(-6.2pp); 即時預訂 <= 0.00(-5.3pp)

LIME 是什麼?(白話版)

模型是一個超級複雜的黑盒子,有 500 棵決策樹,沒人看得懂它整體在想什麼。LIME 的想法是:**我不解釋整個模型,我只解釋「這一間房」附近發生什麼事。**

做法像這樣:

1. 以這間房為中心,亂數產生 2000 個「長得很像它、但數值稍微不同」的假房源
2. 把這 2000 個假房源全部丟給模型,收集模型給的分數
3. 用一條**簡單的直線公式**去逼近這 2000 個點
4. 這條直線的係數,就是「在這間房附近,每個特徵影響多大」

就像用一把直尺去量一段彎曲的路——整條路是彎的,但**只看你腳下這一小段**,用直尺量就夠準了。

數字怎麼讀:

顯示	意思
+11.1 pp	這個條件讓「高風險機率」 增加 11.1 個百分點
-6.2 pp	這個條件讓機率 減少 6.2 個百分點(加分項)
紅色長條長度	相對於最大值的比例(視覺化用)

「**近期平均最短入住 > 20.80**」是什麼? 特徵 `min_nights_avg_ntm` (近期平均最短入住晚數)大於 20.8 晚。這代表這間房實際上偏向長租,短租客訂不到,所以推高空屋風險。

注意 一個容易誤會的細節: 截圖裡使用者把「最低入住天數」調到 30,但 LIME 顯示的是 `近期平均最短入住` (`min_nights_avg_ntm`)——**這是兩個不同的特徵**:

- `minimum_nights`:目前設定的最低入住天數(What-if 可以改)
- `min_nights_avg_ntm`:Airbnb 記錄的「近期實際平均」最短入住(歷史資料,What-if 不會改)

所以調滑桿不會改變這一條 LIME 原因。這是資料本身的結構,不是 bug。

程式: `modules/lime_explainer.py` (97 行) **分類:** ● **推估** 模型推估的解釋(**解釋的是模型,不是現實**)

注意 **最重要的但書:**LIME 解釋的是「**模型為什麼這樣想**」,不是「**現實中什麼導致空屋**」。如果模型本身學錯了,LIME 會忠實地把錯誤的邏輯解釋給你聽。它是模型的翻譯機,不是真相偵測器。

3.6.3 右側:改善建議(截圖 12 右下)

● **推估** 觀察層:以下為規則引擎建議;LLM 個人化建議於 ● 紅色高風險時才可手動產生。

1. 入住天數限制偏嚴:近期最短入住設定偏長,建議放寬以承接短住需求。
依據:此因素推高高風險機率 +11.1 個百分點(LIME 局部歸因)
2. 設施低於周邊競品標配:建議優先增加:廚房/可開伙(周邊 64% 競品有)、
工作空間(周邊 58% 競品有)。
依據:與 1km 內 835 筆跨平台競品之設施覆蓋率比較
3. 最低入住天數過長:建議在法規允許內放寬至 1~2 晚,承接週末短住需求。
依據:目前最低入住 30 晚

建議是怎麼產生的?(規則引擎,四種來源依優先序)

優先序	來源	觸發條件
1	LIME 歸因 Top-2	該特徵有對應的建議模板
2	跨平台價格落點	貴於同層競品 $\geq 60\%$
3	設施缺口	周邊 $\geq 50\%$ 競品有、你沒有
4	規則兜底	最低入住 ≥ 3 晚、描述字數 < 200 字

建議文字是預先寫好的模板,不是 AI 生成的。例如:

```
"minimum_nights": ("最低入住天數過長",  
"過嚴的入住天數限制會流失短住旅客,建議在法規允許內放寬至 1~2 晚。")
```

程式: `modules/suggestion.py` (SHAP_TEMPLATES 字典 + `suggest()` 方法) **分類:** ● **推估** 歸因 + ● **統計** 競品統計 + 固定模板

LLM 建議何時出現? 只有 ● 紅色高風險(機率 $\geq 60\%$)時才會出現「產生 LLM 智慧建議」按鈕,而且**必須手動按**。設計理由:

1. 控制 API 成本(不是每間房都打)
2. 避免每拖一次滑桿就呼叫一次

LLM 拿到什麼材料:模型分數 + LIME 原因 + 競品落點 + 設施缺口。**所有數字都是程式算好的**,LLM 只負責組織成通順的行動建議。詳見 [doc/07_LLM生成原理說明.md](#)。

3.6.4 趨勢線:What-if 模擬 × 真實市場對照(截圖 13)

這張圖有**三條線**,分別代表完全不同的東西:

線	顏色	意思	分類
藍色實線	藍	模型對「如果調到這個價格」的預測空屋率	● 推估
黃色虛線	黃	模型對「如果調到這個價格」的高風險機率	● 推估
菱形虛線	紫	真實市場 :同區同房型房源在各價格帶的 實際 平均空屋率	● 真實

菱形線怎麼算的(這是這張圖最有價值的部分):

第一步: 找同區同房型的房源 (截圖顯示 n=196)
 第二步: 把價格範圍切成 8 段
 第三步: 每一段, 只有樣本 ≥ 5 筆才畫點
 第四步: 該點的高度 = 那一段房源的「實際 Y_vacancy」平均
 菱形大小 = 樣本數 (越大越可信)

這張圖在回答什麼問題?

「模型的 What-if 模擬,跟真實市場的表現一致嗎?」

如果藍線和菱形線**趨勢一致**(都是價格越高空屋率越高),代表模擬可信。截圖顯示兩者確實都往右上走 → **模擬方向合理**。

其他標記:

- 紫色垂直虛線 = 目前定價 \$1,229
- 紅色水平虛線 = 60% 高風險線
- hover 提示 = 「(1,007.375, 58.2052) 價格帶均值 • 49 筆真實房源」

程式: `pages/1__房東入口.py` 第 447~497 行

這張圖是我覺得整個平台最誠實的一張:它把「模型的猜測」和「真實市場數據」畫在同一張圖上,讓使用者自己判斷模型可不可信,而不是只丟一條線出來要人相信。

3.7 分頁六:通知中心

截圖 對應你截圖 14~15

程式: `modules/notify_center.py` (356 行)

3.7.1 觸發條件(兩個條件「或」的關係)

條件 A (風險): 高風險機率 $\geq$ 門檻 (滑桿, 預設 0.60)
 條件 B (空檔): 未來 30 天內最長連續空檔 $\geq$ 門檻 (滑桿, 預設 14 天)

任一成立 → 進入通知清單

為什麼空檔用 30 天窗口而不是 90 天? 早期版本用 90 天,結果 63% 的房源都被觸發 —— 因為遠期日曆多半還沒開放,「空檔」滿地都是。改成 30 天後降到 28%,因為近期日曆已經開放,空檔才是真訊號。**這是實測後修正的決定,不是拍腦袋。**

3.7.2 四張統計卡(截圖 14)

本房東觸發 24 間
觸發原因分布 風險 0 (空檔 24 · 兩者 0)
全平台觸發 2,184 間 (占比 37%)
已處理 0 間

我實際驗算的結果:

數值	驗算	結果
本房東觸發 24 間	該房東 81 間中, <code>gap_longest_30d ≥ 14</code> 的有 24 間	可用
風險 0	該房東 81 間的機率 最大值只有 0.421 ,沒有任何一間 ≥ 0.60	可用
空檔 24	24 間全部由空檔條件觸發	可用
全平台 2,184 間	5,849 間中符合任一條件的	可用
占比 37%	$2,184 \div 5,849 = 37.3\%$	可用

「風險 0」不是壞掉了 — 它正確反映了一件事:這位房東的房源在模型眼中都不算高風險,但檔期上有 24 間正在流血。這正是「隱形危機」象限存在的理由。

3.7.3 處理優先序(截圖 15)

處理優先序(體質 × 檔期象限): **隱形危機** 隱形危機 19 間 | **真警報** 真警報 5 間。
清單已依此排序, **真警報** 真警報(體質差且檔期空)排最前。
檔期 其中 24 間是僅由空檔條件觸發 — 風險分數未達標,但未來 90 天有連續 14 天
以上無訂單,屬於「模型看不出來、但檔期已經在流血」的房源。

排序邏輯:象限優先序 → 機率高低 → 最長空檔天數

程式: `notify_center.py` 第 226~230 行

3.7.4 通知清單每一列

#1356177442322068221 #GNstay公館棧 中山國小一館 雙人獨立套房 有大樓管
● **推估** 觀察 40% **檔期** 空檔 **真警報** 真警報 中山區·空屋率 65%· **檔期** 30天空檔 29 天(最長 29 天)
[手動寄出] [**可用** 已處理]

元素	來源	分類
房源 ID / 名稱	<code>listings_cleaned</code>	● 真實
● 推估 觀察 40%	模型機率與等級	● 推估
檔期 空檔	觸發原因標籤	● 真實
真警報 真警報	四象限分類	● 推估 + ● 真實
空屋率 65%	<code>vac_pred</code>	● 推估
30天空檔 29 天(最長 29 天)	<code>gap_days_30d</code> / <code>gap_longest_30d</code>	● 真實

「**手動寄出**」會做什麼:產生一封模擬信件(**不會真的寄出**),內容包含風險數字、空檔明細、智慧建議。收件信箱是用房東名稱產生的假信箱(`fake_host_email()`),因為原始資料沒有 email 欄位。

這是示範功能,不是真的郵件系統。

3.8 分頁七:月報

按「產生月報」後,程式把前面所有分頁的分析彙整成一份 Markdown 文件,可下載成 `.md` 或 `.html` (用瀏覽器列印可存成 PDF)。

內容章節:風險等級 → 檔期進度 → 空檔明細 → 營收最適定價 → 評論面向 → AI 摘要

「**AI 摘要**」有兩種:規則摘要(預設)或 LLM 摘要(勾選才用,需要金鑰)。

程式: `modules/report_builder.py` (287 行)

第 4 章 • 租客入口

程式: `pages/2__租客入口.py` (792 行)

這頁是給找房的人用的,核心是「依你重視的生活機能排序房源」。

4.1 側邊欄:機能權重滑桿

使用者可以調整 7 種生活機能的重要性(捷運、公車、超商、餐廳、公園、學校、診所)。

4.2 生活機能分數怎麼算(本頁最核心的公式)

每一種設施類型都會得到一個 0~10 分,由兩個部分組成:

第一部分:接近度(proximity)— 最近的一家有多近

$\text{prox} = 10 \times \max(0, 1 - \text{最近距離} \div 1200)$

白話:1200 公尺外得 0 分,就在門口得 10 分,中間線性遞減。

例:最近超商 300 公尺 $\rightarrow 10 \times (1 - 300/1200) = 7.5$ 分

第二部分:密度(density)— 附近有幾家

$\text{dens} = 10 \times \min(1, \log(1+\text{數量}) \div \log(1+25))$

白話:800 公尺內的數量,用對數換算。

為什麼用對數?因為從 0 家變 1 家是天壤之別,

從 20 家變 21 家幾乎沒感覺。對數正好能表達這種「邊際遞減」。

25 家封頂(再多也是 10 分)。

最後合併:

該類型分數 $= 0.6 \times \text{prox} + 0.4 \times \text{dens}$

為什麼接近度佔 6 成、密度佔 4 成? 因為「有沒有」比「有幾家」重要——走 5 分鐘有一家超商,比走 20 分鐘有十家超商實用。

加權總分:

加權分數 $= \Sigma(\text{各類型分數} \times \text{使用者設定的權重})$

百分比 $= \text{加權分數} \div (10 \times \text{權重總和}) \times 100\%$

程式: `modules/geo_utils.py` 的 `convenience_score()` 第 245~267 行 分類:●統計 統計整理(這是一個自訂的評分公式,不是模型預測)

注意 誠實說明:這個公式裡的 1200 公尺、800 公尺、25 家封頂、0.6/0.4 權重,都是團隊自己訂的,不是從資料學出來的,也沒有經過使用者研究驗證。它是一個合理但主觀的設計。換一組參數,排序就會改變。

名詞小教室:為什麼要用「對數」? 對數就是「壓縮大數字」的工具。你手上有 100 元時多賺 100 元很開心;有 100 萬時多賺 100 元沒感覺。這種「越多越無感」的現象,用對數表達最自然。這裡用它來避免「附近有 50 家超商」的地方分數暴衝。

4.3 上方四張 KPI

數值	公式	分類
符合房源	通過篩選條件的房源數	● 真實
中位價格	這些房源價格的中位數	● 統計
平均評分	<code>review_scores_rating</code> 的平均	● 統計
平均機能分	<code>conv_pct</code> 的平均	● 統計

4.4 房源詳情:AI 照片清晰度

點開任一房源會看到「AI 照片清晰度:清晰 / 尚可 / 模糊」+ 清晰機率。

怎麼判斷的(沒有用深度學習):

第一步:下載照片,轉成灰階

第二步:算 5 個影像指標

- ① Laplacian 變異數 — 最關鍵的失焦偵測指標
- ② 解析度(百萬像素)
- ③ 亮度(是否適中)
- ④ 對比
- ⑤ 邊緣密度

第三步:丟進一個「可解釋的線性公式」

分數 = $-3.0 + 3.0 \times \text{清晰度} + 0.8 \times \text{解析度} + 0.9 \times \text{亮度} + 0.7 \times \text{對比} + 1.0 \times \text{邊緣密度}$

機率 = $\text{sigmoid}(\text{分數})$

第四步:分級

機率 $\geq 0.60 \rightarrow$ 清晰

機率 $\geq 0.40 \rightarrow$ 尚可

其餘 $\rightarrow$ 模糊

名詞小教室:Laplacian 變異數是什麼? 它在測量「照片裡的邊緣有多銳利」。清楚的照片,物體邊緣是「咻」一下的明暗跳變 $\rightarrow$ 變異數大 模糊的照片,邊緣是「糊」成一片的漸層 $\rightarrow$ 變異數小 這是攝影界公認的經典失焦偵測法,不需要 AI。

SHAP 貢獻怎麼算的:因為這是線性公式,SHAP 值有精確解,不需要近似:

$$\phi_i = \text{權重}_i \times (\text{實際值}_i - \text{基準值}_i)$$

程式: `modules/image_analysis.py` (241 行) **分類:** ● 統計 影像指標 + 自訂公式

注意 誠實說明:那組權重(3.0、0.8、0.9、0.7、1.0)和基準值,是人工設定的,不是訓練出來的。它是一個「可解釋、可離線跑、不吃記憶體」的替代方案。程式裡也留了 CLIP 深度模型的接口,但預設關閉(Streamlit 免費版記憶體不夠)。

4.5 便利性雷達圖 / 周邊機能明細 / 周遭地圖

區塊	數值來源	分類
雷達圖	7 種機能的 0~10 分	● 統計
機能明細表	1 公里內各類型數量、最近地點、最近距離	● 真實
周遭地圖	座標 + 推估地址 + Haversine 距離	● 真實 + ● 統計

4.6 NLP 評論摘要

數值	怎麼算	分類
平均情感分數	英文用 VADER 套件、中文用自訂正負詞典	● 統計
高頻正/負面詞	斷詞後計數,取前 N 名	● 統計
近期評論片段	直接顯示原文	● 真實

程式: `modules/nlp_analysis.py` (294 行)

第 5 章 • 後台分析

程式: `pages/3_後台分析.py` + `modules/vacancy_model.py` + `modules/backend_v2_sections.py` + `modules/portfolio_sections.py`

這頁是給平台營運方看的,偏研究性質。

5.1 上半部:策略沙盒

5.1.1 左欄:唯讀歷史資料 + 可調參數

區塊	內容	可否調整
唯讀歷史	行政區、性價比評分、清潔度評分、溝通評分	不可用
周邊機能快照	最近捷運/超商/餐廳/公園、評論情感得分	不可用
動態參數	每晚房價、最低入住天數、客服回覆速度、描述文案	可用

5.1.2 右欄:半圓儀表

數值	公式	分類
儀表百分比	模型 A 預測的空屋率	● 推估
低/中/高風險	<40% / 40~70% / ≥70%	● 推估
P(空屋率 ≥ 60%)	模型 B 校準機率	● 推估
紅/黃/綠警報	0.60 / 0.35 門檻	● 推估

注意 注意:這頁的儀表用的是「即時推論」(把當前參數丟給模型現算),**不是 OOF 誠實預測**。所以這裡的數字會比房東入口的略微樂觀。房東入口刻意統一用 OOF,後台沙盒則是為了讓參數調整有即時反應。

5.1.3 貢獻度瀑布圖

每個特徵的貢獻 = 用「現在的值」預測的機率 - 用「全體中位數」預測的機率

白話:「如果這間房的這個特徵變成大眾水準,風險會差多少?」正值(紅、向右)= 推高風險,負值(綠、向左)= 降低風險。

程式: `modules/vacancy_model.py` 的 `contributions()` 第 250~270 行 分類:● 推估 模型推估的解釋

注意 這不是嚴格的 SHAP,是「單一特徵替換成中位數」的邊際貢獻。計算快、好懂,但不滿足 SHAP 的可加性公理。真正的 SHAP 在下方的「SHAP 可解釋性」分頁。

5.2 模型評估分頁(最重要的誠實揭露)

5.2.1 模型訓練流程

資料:dataset_multimodal.csv, 5,849 間房 × 59 特徵
標籤:Y_high_risk = (Y_vacancy ≥ 0.6), 高風險占比 37.4%

模型 A(迴歸):LightGBM, 預測空屋率 0~1
模型 B(分類):LightGBM + Isotonic 校準, 預測 P(空屋率 ≥ 0.6)
對照組:XGBoost(同樣兩軌)
基準線:線性迴歸 + 邏輯迴歸

評估:GroupKFold(host_id) 5 折 OOF(誠實)
+ 單次隨機切分(樂觀, 對照展示用)

59 個特徵的組成:

類別	數量	例子
結構化	13	可住人數、臥室數、價格、最低入住天數、房型、行政區
評分	7	總體評分、清潔度、溝通、地點、性價比
競爭	5	同區價格百分位、同區評分百分位、1km 房源密度
房東身分	7	接受率、回覆率、回覆速度、超讚房東、經營天數
經營用心度	4	描述字數、自介字數、周邊介紹字數、設施總數
地點/房間	6	1km 飯店數、每人單價、每房單價
POI	11	最近捷運距離、500m 公車站數、200m 超商數、最近公園距離
NLP	4	評論情感得分、評論平均字數、無評論、負評比例
座標	2	緯度、經度

5.2.2 模型成績單(data/_model_metrics.json 實際數值)

```
{
  "n": 5849,
  "n_features": 59,
  "high_risk_rate": 0.374,
  "R2": 0.243,
  "MSE": 0.0724,
  "AUC": 0.714,
  "Recall": 0.270,
  "F1": 0.387
}
```

每個指標白話解釋:

指標	數值	白話意思	滿分/隨機
AUC	0.714	隨機抓一間真高風險和一間非高風險,模型把高風險的分數排得比較高的機率是 71.4%	隨機猜 = 0.5,完美 = 1.0
R²	0.243	空屋率的變化,模型只解釋了 24.3%,剩下 75.7% 是模型不知道的	完美 = 1.0
精確率@0.6	約 0.70	模型說「高風險」的房源,70% 真的是	—
召回率@0.6	0.270	真正的高風險房源中,只抓到 27%	—
F1	0.387	精確率與召回率的調和平均	—

5.2.3 前瞻驗證(最誠實、也最刺眼的一節)

這是本專案我認為最有價值的部分:用 9 個月後的真實資料,去檢驗當初訓練的模型。

模型訓練用的房源資料: 2025-09-30

用來驗證的真實日曆: 2026-06-30

中間相隔: 9 個月

比對成功的房源: 4,227 間

結果(`models/forward_validation.json` 實際數值):

指標	交叉驗證(OOF)	真實未來	掉了多少
分類 AUC	0.714	0.632	-0.082
迴歸 R ²	0.243	0.003	幾乎歸零
紅色層精確率	約 0.70	0.570	-0.13
紅色層召回率	0.270	0.205	-0.065
預測空屋率 vs 真實空屋率相關係數	—	0.289	—

這些數字白話翻譯:

- **AUC 0.632**:比亂猜(0.5)好,但好得有限。約略等於「模型排序有點用,但不太可靠」。
- **R² 0.003**:幾乎等於零。意思是模型預測的空屋率數值,對真實空屋率幾乎沒有解釋力。
- **精確率 0.570**:模型說高風險的 603 間房裡,只有 57% 真的高風險 —— 接近擲硬幣。

這代表什麼?

模型的絕對數值(「空屋率 65%」)不能信。模型的相對排序(「這間比那間風險高」)還有點參考價值。真正可靠的是日曆資料(● 真實)。

這就是為什麼整個平台的設計是「真實資料當主角、模型當配角」—— 四象限、通知中心的空檔觸發、定價情報的真實已訂率基準,全部是為了補模型的不足。

為什麼模型會退化這麼多? 三個原因:

1. **時間差 9 個月**:市場、季節、競爭都變了
2. **標籤本身的定義有問題**: `Y_vacancy = availability_365 ÷ 365` 這個「未來一年可訂天數」,對遠期的部分本來就受「房東還沒開放日曆」污染

3. **訊號天花板**: doc/01 的實驗顯示,五種模型(LogReg / RF / HGB / XGB / LGBM)的 AUC 全部卡在 0.720.73,加上 POI 和 NLP 特徵只提升 0.0020.005。瓶頸在資料本身,不是模型不夠強。

5.3 SHAP 可解釋性分頁

這裡才是真正的 SHAP(TreeExplainer,對 500 棵樹做精確計算),取 500 筆樣本做全域解釋。

名詞小教室:SHAP 和 LIME 有什麼不同? 兩者都在回答「哪個特徵影響最大」,但角度不同:

- **SHAP**:來自賽局理論,回答「如果這些特徵是一個團隊,每個人該分多少功勞?」有嚴謹的數學保證(可加性),但計算慢。
- **LIME**:回答「在這一間房附近,模型的行為像什麼樣的簡單直線?」計算快,但只在局部有效。

本平台的分工:**房東入口用 LIME**(要快、要單筆解釋),**後台用 SHAP**(要嚴謹、要全域趨勢)。

5.4 市場獲利地圖分頁

數值	公式	分類
年營收估算	每晚價格 × 未來 365 天真實已訂天數	● 真實 + ● 統計
各房型中位年營收	依房型分組取中位數	● 統計
行政區 × 房型矩陣	交叉分組取中位數	● 統計
長短租策略分析	依日曆的每日最低入住天數判定策略,比較各策略的填充率與營收	● 統計

注意 同樣的限制:「已訂天數」受 `available='f'` 二義性影響,封鎖日曆的房源會被誤算成高營收。

5.5 評論面向分析(全市)

數值	公式	分類
分析評論則數	各面向提及次數加總	● 真實
最大痛點	負評率最高的面向	● 統計
最強優勢	負評率最低的面向	● 統計
行政區 × 面向熱力圖	交叉分組的負評率	● 統計

5.6 通知中心(全平台)

與房東入口的通知中心是**同一個模組**,差別只在 `host_id=None` (顯示全平台而非單一房東)。

第 6 章・名詞小字典(依首字筆畫)

名詞	一句話解釋
AUC	模型排序能力的分數。0.5 = 亂猜,1.0 = 完美。本平台前瞻 AUC 0.632。
BallTree	快速找「附近的點」的資料結構。把地圖切成一層層的球,不用逐點計算距離。
F1	精確率與召回率的平衡分數,兩者都高才會高。
GroupKFold	交叉驗證的一種,確保同一個房東的房源不會被拆到訓練和測試兩邊,防止作弊。
Haversine	算地球表面兩點距離的公式(因為地球是圓的,不能用平面直線)。
Isotonic 校準	把模型「過度自信」的機率調正,讓它說 70% 時真的約 70% 準。
Laplacian 變異數	測照片邊緣有多銳利,用來判斷失焦。
LightGBM / XGBoost	兩種主流的「梯度提升樹」演算法,用幾百棵決策樹投票。
LIME	用一條簡單直線去逼近黑盒模型在「某一筆資料附近」的行為,藉此解釋單筆預測。
OOF(袋外預測)	每筆資料的預測,都來自「訓練時沒看過它」的模型。誠實分數。
pp(百分點)	60% 變 66% 是「+6 個百分點」,不是「+6%」。
qcut(等頻分箱)	分組時讓每組的樣本數一樣多,而不是讓每組的範圍一樣寬。
R ²	模型解釋了資料變化的百分之幾。0 = 完全沒解釋力,1 = 完美。
SHAP	用賽局理論分配「功勞」給每個特徵,有嚴謹數學保證。
中位數	排隊站中間那個人的值。不會被極端值拉走。
百分位	排名的百分比版。「貴於 39%」= 比 39% 的競品貴。
召回率	真正有問題的裡面,你抓到了幾成。
每人每晚	跨平台比價的統一單位 = 每晚等效價 ÷ 可住人數。
空屋率(Y_vacancy)	未來一年可被預訂的天數 ÷ 365。
面向情感分析(ABSA)	把一則評論拆成多個面向分別判斷好壞,而不是整篇算一個分數。
精確率	你說有問題的裡面,幾成是真的。
樹模型的階梯性	決策樹用「大於某值嗎」判斷,所以輸出是一階一階跳的,微調參數常常沒反應。
雙層警報	紅色門檻 0.60(求準)、黃色門檻 0.35(求全)。

第 7 章・誠實限制總表

這一章把全文散落的限制集中起來。**面試或發表被問到時,直接看這張表。**

7.1 資料層面

#	限制	影響哪些數字	嚴重程度
1	available='f' 分不出「被訂走」和「房東封鎖」	所有訂房率、空檔、營收估算	● 高
2	房源資料與日曆相差 9 個月	模型的所有預測	● 高
3	遠期日曆多未開放	365 天訂房率、逐月比較	● 高(已在 3.5.3 節處理)
4	兩批資料房源清單對不上,約 1,600 間無檔期資料	四象限的「資料不足」類別	● 推估 中
5	5,849 筆中有 701 筆(12%)沒有價格、1,632 筆(28%)沒有房源名稱	卡片、地圖、月報的顯示	● 推估 中(已加缺值處理)
6	591/Booking/DD 的爬取時間(2026-07)與 Airbnb(2025-09)不同	跨平台比價	● 推估 中
7	Airbnb 不公開門牌,地址是用最近超商反推的	詳情頁與地圖的地址	● 真實 低
8	0 間房源有合法登記字號	整個市場的合法性	● 高(業務風險)

7.2 方法層面

#	限制	說明
9	模型前瞻 R^2 僅 0.003	預測的空屋率數值幾乎沒有解釋力,只有排序有參考價值
10	五種模型 AUC 全卡在 0.72~0.73	瓶頸在資料訊號,不是模型不夠強
11	營收最適定價是橫斷面推論	不是因果關係,且有「幾乎總是叫你漲價」的系統性偏誤
12	跨平台換算未計押金、管理費、最短租期	只能看相對定位
13	ABSA 是關鍵詞比對,不是深度學習	會誤判否定句(「不會吵」含「吵」)
14	照片清晰度的權重是人工設定的	不是訓練出來的
15	生活機能分數的參數是主觀設定的	1200m、800m、25 家封頂、0.6/0.4 權重都沒有經過驗證
16	設施是從文字描述比對的	「有但沒寫」會被判定成「沒有」
17	LIME 解釋的是模型,不是現實	模型學錯了,LIME 會忠實解釋錯誤的邏輯
18	後台沙盒用即時推論,非 OOF	數字會比房東入口略樂觀

7.3 這些限制怎麼被處理

處理方式	對應限制
四象限:模型與檔期衝突時以檔期為準	2, 9, 10
空檔警示改用 30 天窗口	3
逐月比較加上「可判讀」判定	1, 3
排除全年封鎖 / 全年全空的房源	1
面向提及少於 3 次不下結論	13
同層競品少於 5 筆不算價格落點	6
同商圈同房型少於 10 間不建基準	—
全站統一用 OOF 誠實預測	9
缺值一律顯示「價格未提供 / 房源 #ID」	5
畫面上直接標註「模型推估・僅供參考」	9, 10

第 8 章 • 程式檔案對照表

8.1 頁面

檔案	行數	職責
index.py	150	動畫首頁 + 統計列
pages/1__房東入口.py	778	房東營運面板(7 分頁)
pages/2__租客入口.py	792	租客找房 + 生活機能排序
pages/3__後台分析.py	269	營運方後台 + 研究分頁

8.2 分析模組

檔案	行數	職責	產出哪些畫面數字
<code>modules/calendar_analytics.py</code>	~290	檔期分析	訂房率、空檔、營收曲線、逐月基準
<code>modules/quadrant.py</code>	102	體質×檔期四象限	真警報/隱形危機/靠降價撐住/健康
<code>modules/competitor.py</code>	81	跨平台競品索引	平台筆數、中位價、價格落點、設施覆蓋率
<code>modules/market_data.py</code>	185	四平台資料清洗	每人每晚等效價、容量層
<code>modules/feature_engineering.py</code>	215	特徵層 + 預測 + What-if	空屋率、機率、警報等級
<code>modules/lime_explainer.py</code>	97	LIME 解釋	風險原因 Top-3 與權重
<code>modules/suggestion.py</code>	~115	規則建議引擎	改善建議與依據
<code>modules/llm_advisor.py</code>	123	LLM 建議	個人化行動建議
<code>modules/review_summary.py</code>	154	評論摘要	一段話摘要
<code>modules/absa_sections.py</code>	273	面向情感 UI	負評率長條圖、市場熱力圖
<code>modules/geo_utils.py</code>	313	地理計算	距離、POI 數量、推估地址、機能分數
<code>modules/image_analysis.py</code>	241	照片品質	清晰度標籤與機率
<code>modules/nlp_analysis.py</code>	294	情感與關鍵詞	情感分數、高頻詞
<code>modules/vacancy_model.py</code>	371	後台雙軌模型	儀表、貢獻度瀑布
<code>modules/notify_center.py</code>	~360	通知中心	觸發統計、通知清單
<code>modules/map_view.py</code>	265	Leaflet 互動地圖	點位、圖層、距離清單
<code>modules/report_builder.py</code>	~295	月報生成	Markdown / HTML 報告
<code>modules/ui_components.py</code>	~360	樣式與共用元件	色票、表格、安全格式化

8.3 離線腳本

檔案	行數	執行指令	產出
<code>scripts/train_backend_models.py</code>	386	<code>python -X utf8 scripts/train_backend_models.py</code>	模型、預測、競品索引、成績單
<code>scripts/build_calendar_features.py</code>	201	<code>python -X utf8 scripts/build_calendar_features.py</code>	檔期指標、市場基準、前瞻驗證
<code>scripts/build_absa.py</code>	202	<code>python -X utf8 scripts/build_absa.py</code>	面向情感三個檔案

附錄 · 快速查詢:這個數字是哪一種?

你在畫面上看到	分類	可以拿來做決策嗎
名下房源、評論則數、競品筆數	● 真實	可用 可以,就是數量
近 30/60/90 天訂房率	● 真實	可用 可以,這是真實日曆
空檔天數、最長連續空檔	● 真實	可用 可以
逐日訂房熱度圖	● 真實	可用 可以,最原始的資料
距離、最近捷運/超商	● 真實	可用 可以
空檔機會成本(金額)	● 統計	注意 這是上限,不是實際損失
同商圈中位、排名、百分位	● 統計	可用 可以看相對位置
每人每晚、價格落點	● 統計	注意 只能看相對定位(未計押金等)
設施覆蓋率、缺口	● 統計	注意 「有但沒寫」會誤判
面向負評率	● 統計	注意 提及 <3 次不算數
營收最適定價	● 統計	注意 橫斷面推論,且傾向叫你漲價
生活機能分數	● 統計	注意 公式參數是主觀設定的
照片清晰度	● 統計	注意 權重是人工設定的
預測空屋率	● 推估	不可用 絕對數值不可信(前瞻 $R^2=0.003$)
高風險機率、警報等級	● 推估	注意 只能當 排序 參考(前瞻 AUC 0.632)
LIME 原因、貢獻度	● 推估	注意 解釋的是模型,不是現實
四象限分類	● 推估 + ● 真實	可用 推薦使用 — 這是唯一結合兩種來源的判斷

本文所有數值都經過原始資料重新計算驗證。若程式更新,請以程式碼為準。